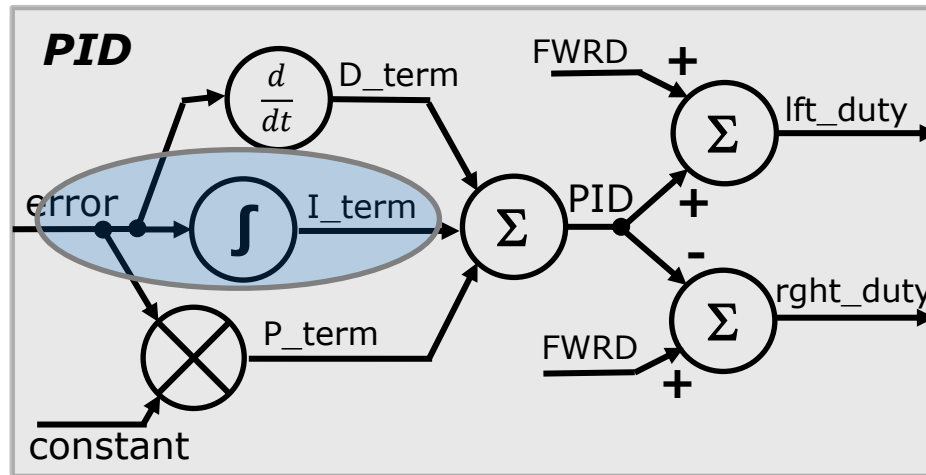
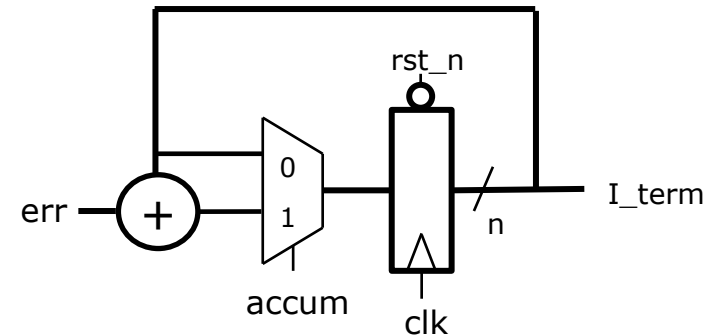




Exercise 08: I Term Datapath



- Integration is nothing more than summing over time, so this is implemented with an accumulator.

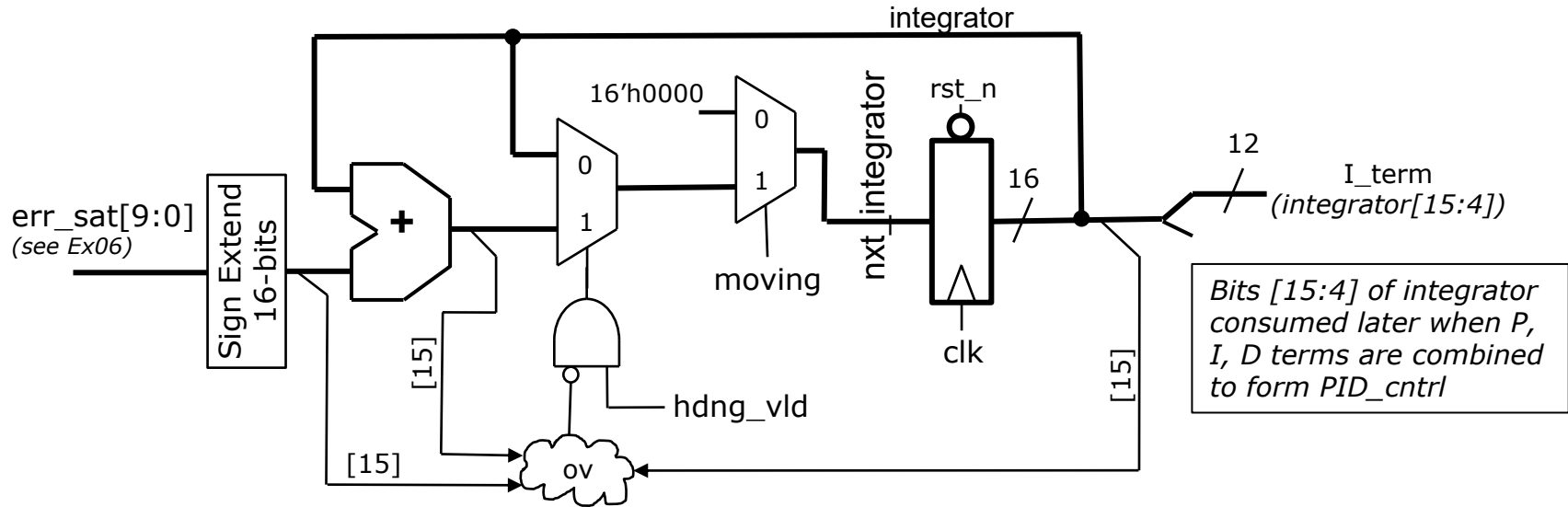


Design considerations:

- We don't want to accumulate every clock cycle (50MHz clk on DE0-nano)
→ On every valid (**hdng_vld**) reading, the saturated error (**err_sat**) is accumulated into a 16-bit accumulator.
- MazeRunner can only correct its error when it is **moving**.
→ Zero out accumulator if it's not moving
- We want a saturating accumulator so that it doesn't roll over from +ve to -ve (or vice versa)
→ Accumulate only if it will not overflow



Exercise 08: I Term Datapath



Signal	Dir	Description
clk, rst_n	in	System clock and asynch active low reset
hdng_vld	in	A new error signal is valid and should be accumulated into I_term
moving	in	The MazeRunner is moving so PID should be active
err_sat[9:0]	in	The error of the course (heading – desired_heading)
I_term[11:0]	out	The I_term for eventual use in PID control



Exercise 08: I Term Datapath

- Implement **I_term.sv**
- The accumulator register should be inferred with a simple **always_ff** block that infers a 16-bit wide register that is asynchronously reset and takes on the value of **nxt_integrator**.
 - **nxt_integrator** will be a 16-bit wide internal signal that you generate using **assign** statements.
- Create a self-checking testbench (**I_term_tb.sv**)
- Initially assert **rst_n**, then deassert it (goes high) on the first negative edge of clk (see example code).
- Test various scenarios involving **moving**, **hdng_vld**, and **ov** to ensure functionality

```
always_ff @(posedge clk, negedge rst_n)
  if (!rst_n)
    integrator <= 16'h0000;
  else
    integrator <= nxt_integrator;
```

```
initial begin
  clk = 0;
  rst_n = 0;
  moving = 1;
  hdng_vld = 1;
  err_sat = 10'h1FF;
  @(negedge clk);
  rst_n = 1; // deassert reset

  repeat (10) @(posedge clk);
  ...
end

always
  #5 clk = ~clk;
```